

模块五：矩阵快速幂与递推优化

一、Fibonacci

问题：求 Fibonacci 数列第 n 项, $n \leq 10^{18}$ 。

递推式： $F_n = F_{n-1} + F_{n-2}$, $F_1 = F_2 = 1$ 。

核心思想：将递推写成矩阵形式，用快速幂加速。

$$\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \times \begin{bmatrix} F_{n-1} \\ F_{n-2} \end{bmatrix}$$

迭代 $n - 2$ 次：

$$\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-2} \times \begin{bmatrix} F_2 \\ F_1 \end{bmatrix}$$

模板代码 (2×2 矩阵)：

```
struct Mat {
    long long a[2][2];
    Mat() { memset(a, 0, sizeof(a)); }
};

Mat mul(Mat A, Mat B, long long mod) {
    Mat C;
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)
            for (int k = 0; k < 2; k++)
                C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j]) % mod;
    return C;
}

Mat mpow(Mat base, long long exp, long long mod) {
    Mat res;
    res.a[0][0] = res.a[1][1] = 1; // 初始化为单位阵
    while (exp) {
        if (exp & 1) res = mul(res, base, mod);
        base = mul(base, base, mod);
        exp >>= 1;
    }
}
```

```
    return res;
}
```

复杂度： $O(\log n)$, 2×2 矩阵乘法常数极小。

二、Fibonacci

除了矩阵快速幂，Fibonacci 还有一种**倍增算法**（常数更小，代码更短）：

利用恒等式：

$$F_{2k} = F_k \times (2F_{k+1} - F_k)$$

$$F_{2k+1} = F_{k+1}^2 + F_k^2$$

递归计算 F_n ，每次将规模减半：

```
void fib(long long n, long long &fn, long long &fn1) { // {F_n, F_{n+1}}
    if (n == 0) {
        fn = 0, fn1 = 1;
        return;
    }
    long long a, b;
    fib(n >> 1, a, b); // a = F_k, b = F_{k+1}
    long long c = a * (2 * b - a);
    long long d = a * a + b * b;
    if (n & 1) fn = d, fn1 = c + d;
    else fn = c, fn1 = d;
}
```

对比：倍增算法只用整数运算，无需矩阵乘法，常数更小。但矩阵快速幂的**通用性更强**，能处理任意线性递推。

三、一般线性递推的矩阵构造

核心方法：将递推式写成“状态向量 $\times$ 转移矩阵”的形式。

类型1：齐次线性递推

$$f(n) = c_1 f(n-1) + c_2 f(n-2) + \cdots + c_k f(n-k)$$

状态向量： $[f(n), f(n-1), \dots, f(n-k+1)]^T$

转移矩阵 ($k \times k$) :

$$\begin{bmatrix} c_1 & c_2 & c_3 & \cdots & c_{k-1} & c_k \\ 1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 1 & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 1 & 0 \end{bmatrix}$$

例题: $f(n) = 2f(n-1) + 3f(n-2)$, $f(1) = 1, f(2) = 2$.

$$\begin{bmatrix} f(n) \\ f(n-1) \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix}^{n-2} \begin{bmatrix} f(2) \\ f(1) \end{bmatrix}$$

类型2: 带常数项

$$f(n) = c_1 f(n-1) + c_2 f(n-2) + d$$

增广一维常数 1:

$$\begin{bmatrix} f(n) \\ f(n-1) \\ 1 \end{bmatrix} = \begin{bmatrix} c_1 & c_2 & d \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} f(n-1) \\ f(n-2) \\ 1 \end{bmatrix}$$

类型3: 带多项式项

$$f(n) = f(n-1) + f(n-2) + n^2$$

需要把 n^2 也纳入状态, 利用 $(n+1)^2 = n^2 + 2n + 1$:

状态向量: $[f(n), f(n-1), n^2, n, 1]^T$

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 2 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

四、Fibonacci 常见延伸

延伸1: 广义 Fibonacci

$$G_n = aG_{n-1} + bG_{n-2}, \quad G_1 = x, G_2 = y$$

转移矩阵 $\begin{bmatrix} a & b \\ 1 & 0 \end{bmatrix}$, 初态 $\begin{bmatrix} y \\ x \end{bmatrix}$.

延伸2：带前缀和

求 $S_n = \sum_{i=1}^n F_i$ 。增广一维：

$$\begin{bmatrix} F_n \\ F_{n-1} \\ S_n \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} F_{n-1} \\ F_{n-2} \\ S_{n-1} \end{bmatrix}$$

延伸3：矩阵快速幂求斐波那契前 n 项平方和

恒等式： $\sum_{i=1}^n F_i^2 = F_n \times F_{n+1}$ ，可直接用矩阵求 F_n, F_{n+1} 。

五、模板默写：矩阵快速幂（n*n的矩阵）

习题：

题号	名称
Luogu P1962	【模板】斐波那契数列
Luogu P1939	【模板】矩阵加速（数列）
Luogu P1349	广义斐波那契数列

模块六：筛法延伸**第一部分：线性筛求积性函数****一、什么是积性函数？**

定义：若函数 $f(n)$ 满足：对任意互质的正整数 a, b （即 $\gcd(a, b) = 1$ ），都有

$$f(ab) = f(a) \cdot f(b)$$

则称 f 为**积性函数**。

特别地，若对任意 a, b （不要求互质）都有 $f(ab) = f(a)f(b)$ ，则称为**完全积性函数**。

几乎所有经典积性函数都可以用欧拉线性筛在 $O(n)$ 时间内求出。

二、线性筛的通用框架

线性筛的核心思想：**每个合数只被它的最小质因子筛掉一次**，保证 $O(n)$ 复杂度。

对于积性函数 $f(n)$ ，在线性筛中：

1. **质数**：直接计算 $f(p)$
2. $i \bmod p = 0$ (p 是 i 的最小质因子)：利用 $f(i)$ 和辅助信息推导 $f(i \cdot p)$
3. $i \bmod p \neq 0$ (p 与 i 互质)： $f(i \cdot p) = f(i) \cdot f(p)$

通用代码框架：

```
const int MAXN = 1e6 + 5;
int prime[MAXN], cnt = 0;
bool vis[MAXN];
int f[MAXN];          // 目标函数值
int aux[MAXN];        // 辅助数组（根据具体函数定义）

void linearSieve(int n) {
    f[1] = 1;          // 大部分积性函数 f(1)=1
    for (int i = 2; i <= n; i++) {
        if (!vis[i]) {
            prime[cnt++] = i;
            f[i] = ???;          // 质数处的取值
            aux[i] = ???;        // 辅助信息初始化
        }
        for (int j = 0; 1LL * i * prime[j] <= n; j++) {
            int p = prime[j];
            vis[i * p] = true;
            if (i % p == 0) {
                // p 是 i 的最小质因子
                aux[i * p] = ???;
                f[i * p] = ???;    // 利用 f(i) 和 aux 递推
                break;
            } else {
                // p 与 i 互质
                aux[i * p] = ???;
                f[i * p] = f[i] * f[p];    // 积性直接乘
            }
        }
    }
}
```

三、常见积性函数的线性筛实现

1. 欧拉函数 $\varphi(n)$

定义： $\varphi(n)$ 表示 1 到 n 中与 n 互质的数的个数。

公式： $\varphi(n) = n \times \prod_{p|n} (1 - \frac{1}{p})$

递推：

- 质数 p : $\varphi(p) = p - 1$
- $i \bmod p = 0$: $\varphi(i \cdot p) = \varphi(i) \cdot p$
- $i \bmod p \neq 0$: $\varphi(i \cdot p) = \varphi(i) \cdot (p - 1)$

```
int phi[MAXN];
```

```
void getPhi(int n) {
    phi[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!vis[i]) {
            prime[cnt++] = i;
            phi[i] = i - 1;
        }
        for (int j = 0; j < cnt && 1LL * i * prime[j] <= n; j++) {
            int p = prime[j];
            vis[i * p] = true;
            if (i % p == 0) {
                phi[i * p] = phi[i] * p;
                break;
            } else {
                phi[i * p] = phi[i] * (p - 1);
            }
        }
    }
}
```

2. 莫比乌斯函数 $\mu(n)$

定义：

- $\mu(1) = 1$
- 若 n 有平方因子, $\mu(n) = 0$
- 否则若 n 有 k 个不同素因子, $\mu(n) = (-1)^k$

递推：

- 质数 p : $\mu(p) = -1$
- $i \bmod p = 0$: $\mu(i \cdot p) = 0$ (出现了平方因子)
- $i \bmod p \neq 0$: $\mu(i \cdot p) = -\mu(i)$

```
int mu[MAXN];

void getMu(int n) {
    mu[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!vis[i]) {
            prime[cnt++] = i;
            mu[i] = -1;
        }
        for (int j = 0; j < cnt && 1LL * i * prime[j] <= n; j++) {
            int p = prime[j];
            vis[i * p] = true;
            if (i % p == 0) {
                mu[i * p] = 0;
                break;
            } else {
                mu[i * p] = -mu[i];
            }
        }
    }
}
```

3. 约数个数函数 $d(n)$ (或 $\tau(n)$)

定义: $d(n) = \sum_{d|n} 1$, 即 n 的正约数个数。

若 $n = \prod p_i^{a_i}$, 则 $d(n) = \prod (a_i + 1)$ 。

递推: 需要辅助数组 $cnt[n]$ 记录 n 中最小质因子的指数:

- 质数 p : $d(p) = 2$, $cnt[p] = 1$
- $i \bmod p = 0$: $cnt[i \cdot p] = cnt[i] + 1$,

$$d(i \cdot p) = \frac{d(i)}{cnt[i] + 1} \times (cnt[i \cdot p] + 1)$$

- $i \bmod p \neq 0$: $cnt[i \cdot p] = 1$, $d(i \cdot p) = d(i) \times 2$

```
int d[MAXN], cnt[MAXN]; // cnt[i] 表示 i 中最小质因子的指数
```

```

void getDivisorCount(int n) {
    d[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!vis[i]) {
            prime[cntP++] = i;
            d[i] = 2;
            cnt[i] = 1;
        }
        for (int j = 0; j < cntP && 1LL * i * prime[j] <= n; j++) {
            int p = prime[j];
            vis[i * p] = true;
            if (i % p == 0) {
                cnt[i * p] = cnt[i] + 1;
                d[i * p] = d[i] / (cnt[i] + 1) * (cnt[i * p] + 1);
                break;
            } else {
                cnt[i * p] = 1;
                d[i * p] = d[i] * 2;
            }
        }
    }
}

```

4. 约数和函数 $\sigma(n)$

定义： $\sigma(n) = \sum_{d|n} d$, 即 n 的所有正约数之和。

对 $n = \prod p_i^{a_i}$, 有 $\sigma(n) = \prod \frac{p_i^{a_i+1} - 1}{p_i - 1}$ 。

递推：需要辅助数组 $pw[n]$ 记录 n 中**最小质因子的最高次幂** (即 p^a) :

- 质数 p : $\sigma(p) = p + 1$, $pw[p] = p$
- $i \bmod p = 0$: $pw[i \cdot p] = pw[i] \cdot p$,

$$\sigma(i \cdot p) = \sigma(i) \cdot \frac{pw[i \cdot p] \cdot p - 1}{pw[i \cdot p] - 1}$$

(本质是乘上 $(p^{a+2} - 1)/(p^{a+1} - 1)$)

- $i \bmod p \neq 0$: $pw[i \cdot p] = p$, $\sigma(i \cdot p) = \sigma(i) \cdot (p + 1)$

```
int sigma[MAXN], pw[MAXN]; // pw[i] 表示 i 中最小质因子的最高次幂 p^a
```

```
void getSigma(int n) {
```



```

sigma[1] = 1;
for (int i = 2; i <= n; i++) {
    if (!vis[i]) {
        prime[cntP++] = i;
        sigma[i] = i + 1;
        pw[i] = i;
    }
    for (int j = 0; j < cntP && 1LL * i * prime[j] <= n; j++) {
        int p = prime[j];
        vis[i * p] = true;
        if (i % p == 0) {
            pw[i * p] = pw[i] * p;
            sigma[i * p] = sigma[i] / ((pw[i] * p - 1) / (pw[i] - 1))
                * ((pw[i * p] * p - 1) / (pw[i * p] - 1));
            // 更简洁的写法:
            // sigma[i * p] = sigma[i] * (pw[i * p] * p - 1) / (pw[i * p] -
            break;
        } else {
            pw[i * p] = p;
            sigma[i * p] = sigma[i] * (p + 1);
        }
    }
}
}
}

```

简化写法 (利用递推关系) :

```

// 在 i % p == 0 时
sigma[i * p] = sigma[i] + p * (sigma[i] - sigma[i / p]);

```

这个公式无需 pw 数组, 代码更简洁, 但理解起来稍难。

5. 完全积性函数 (特例)

完全积性函数对任意 a, b 满足 $f(ab) = f(a)f(b)$, 筛法最为简单:

- 恒等函数: $f(n) = 1$
- 幂函数: $f(n) = n^k$
- 刘维尔函数: $\lambda(n) = (-1)^{\Omega(n)}$ ($\Omega(n)$ 为素因子总个数)

递推统一为: $f(i \cdot p) = f(i) \cdot f(p)$, 无需辅助数组。

四、线性筛总结: 一表通

函数	质数 p	$i \bmod p = 0$	$i \bmod p \neq 0$	辅助数组
$\varphi(n)$	$p - 1$	$\varphi(i) \cdot p$	$\varphi(i) \cdot (p - 1)$	无
$\mu(n)$	-1	0	$-\mu(i)$	无
$d(n)$	2	$\frac{d(i)}{cnt[i]+1} \cdot (cnt[i] + 2)$	$d(i) \cdot 2$	$cnt[i]$ (最小质因子指数)
$\sigma(n)$	$p + 1$	$\sigma(i) \cdot \frac{pw[i] \cdot p^2 - 1}{pw[i] \cdot p - 1}$	$\sigma(i) \cdot (p + 1)$	$pw[i]$ (最小质因子最高次幂)

习题:

题号	名称
Luogu U166294	线性筛筛积性函数【模板】

第二部分：区间筛

一、区间筛的基本思想

问题场景：当需要求 $[L, R]$ 区间内的素数或数论函数信息，其中 L, R 很大（如 $R \leq 2^{31}$ 甚至 10^{12} ），但区间长度 $R - L \leq 10^6$ 时，无法直接开数组筛到 R 。

核心思想：

合数 n 必有一个不超过 $\sqrt{n}$ 的质因子。

因此，算法分为两步：

- 1. 先用普通筛法筛出 $[2, \sqrt{R}]$ 内的所有素数
- 2. 用这些素数去标记/分解区间 $[L, R]$ 内的数

复杂度： $O(\sqrt{R} + (R - L) \log \log R)$

二、区间筛通用模板

```

#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

const int MAXN = 1000005;
bool isCompSmall[MAXN];      // 标记 [2, sqrt(R)] 的合数
vector<int> primes;          // 存储 [2, sqrt(R)] 的素数

// 第一步：筛出 [2, sqrt(R)] 的所有素数
void getSmallPrimes(int limit) {
    // 自己用线性筛或者埃氏筛去写
    // 不建议用sqrt单点判断质数的方法去写
}

// 第二步：用这些素数处理区间 [L, R]
// 注意：具体处理逻辑（标记合数/分解质因子/求约数和）因题目而异

```

三、应用

1: 区间筛素数

目标：求出 $[L, R]$ 内的所有素数。

思路：用筛出的小素数标记区间内的合数，未被标记的即为素数。

```

const int MAXN = 1000005;
bool isComp[MAXN];          // isComp[i] = true表示 L+i 是合数

void intervalPrimeSieve(ll L, ll R) {
    memset(isComp, 0, sizeof(isComp));

    int limit = sqrt(R) + 1;
    getSmallPrimes(limit);

    // 用每个小素数标记区间内的合数
    for (int p : primes) {
        // 找到 >= L 的第一个 p 的倍数
        ll start = max(1LL * p * p, (L + p - 1) / p * p);
        for (ll j = start; j <= R; j += p) {
            isComp[j - L] = true;
        }
    }

    // 特判：1 不是素数

```

```

if (L == 1) isComp[0] = true;

// 此时 isComp[i] == false 表示 L+i 是素数
}

```

2: 区间筛质因子个数与约数个数

核心公式

对于 $n = p_1^{a_1} \times p_2^{a_2} \times \cdots \times p_k^{a_k}$:

- 质因子总数 (含重复): $\Omega(n) = \sum a_i$
- 不同质因子个数: $\omega(n) = k$
- 约数个数: $d(n) = \prod (a_i + 1)$

若要求 $d(n^k)$, 则:

$$d(n^k) = \prod (a_i \times k + 1)$$

区间筛质因子个数 (模板)

目标: 求 $[L, R]$ 内每个数的质因子分解信息从而算出 $d(n^k)$ 。

核心思想: 用每个小素数 p 去筛区间内所有 p 的倍数, 边筛边分解, 记录每个质因子的指数。

```

const int MAXN = 1000005;
const ll MOD = 998244353;

ll val[MAXN];    // val[i] 存储 L+i 当前剩余的值 (分解过程中不断除以质因子)
ll cnt[MAXN];    // cnt[i] 存储 L+i 的质因子个数 (或约数个数累乘结果)

void intervalFactorSieve(ll L, ll R, ll K) {
    // K 表示幂次 (求 d(n^K) 时使用, K=1 时就是 d(n))
    int len = R - L;

    // 初始化: val 为原数, cnt 初始为 1 (约数个数公式的累乘起点)
    for (int i = 0; i <= len; i++) {
        val[i] = L + i;
        cnt[i] = 1;
    }

    int limit = sqrt(R) + 1;
    getSmallPrimes(limit);

```

```

// 用每个小素数分解区间内的数
for (int p : primes) {

    // 找到 >= L 的第一个 p 的倍数
    ll start = (L + p - 1) / p * p;
    for (ll j = start; j <= R; j += p) {
        int idx = j - L;
        ll exp = 0;

        // 统计质因子 p 的指数
        while (val[idx] % p == 0) {
            val[idx] /= p;
            exp++;
        }

        if (exp > 0) {
            // 约数个数公式: 乘上 (exp * K + 1)
            cnt[idx] = cnt[idx] * (exp * K + 1) % MOD;
        }
    }
}

// 处理剩余的大质因子 (> sqrt(R))
// 此时若 val[i] > 1, 说明它本身是一个质数 (或质因子), 指数为 1
for (int i = 0; i <= len; i++) {
    if (val[i] > 1) {
        cnt[i] = cnt[i] * (K + 1) % MOD;
    }
}

// 此时 cnt[i] = d((L+i)^K) mod MOD
}

```

代码解读:

1. **val 数组**: 存储每个数的“剩余部分”。每遇到一个质因子就除以它, 直到剩余 1 或一个大质数。
2. **cnt 数组**: 累乘约数个数公式中的 $(a_i \times K + 1)$ 。
3. **最后处理**: 若 $val[i] > 1$, 说明剩下的数是一个大于 $\sqrt{R}$ 的质数, 指数为 1。

复杂度分析: 外层枚举素数 (约 $\sqrt{R}/\log R$ 个), 内层枚举区间内该素数的倍数 (约 $(R - L)/p$ 个)。总复杂度约为 $O((R - L) \log \log R)$ 。

3: 区间筛约数和

核心公式

对于 $n = p_1^{a_1} \times p_2^{a_2} \times \cdots \times p_k^{a_k}$, 约数和为:

$$\sigma(n) = \prod_{i=1}^k \frac{p_i^{a_i+1} - 1}{p_i - 1}$$

若要求 $\sigma(n^K)$, 则每个质因子的指数变为 $a_i \times K$:

$$\sigma(n^K) = \prod_{i=1}^k \frac{p_i^{a_i \times K + 1} - 1}{p_i - 1}$$

区间筛约数和 (模板)

目标: 求 $[L, R]$ 内每个数的约数和 $\sigma(n)$ (或 $\sigma(n^K)$) 。

核心思路: 与筛质因子个数类似, 但累乘的是等比数列求和公式, 而不是指数+1。

```
const int MAXN = 1000005;
const ll MOD = 1e9 + 7;

ll val[MAXN];      // val[i] 存储 L+i 当前剩余的值
ll sigma[MAXN];    // sigma[i] 存储 L+i 的约数和 (累乘结果)

ll qpow(ll a, ll b, ll mod) {
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

void intervalSigmaSieve(ll L, ll R, ll K) {
    // K 表示幂次 (求 sigma(n^K) 时使用, K=1 时就是 sigma(n))
    int len = R - L;

    // 初始化: val 为原数, sigma 初始为 1
    for (int i = 0; i <= len; i++) {
        val[i] = L + i;
        sigma[i] = 1;
    }

    int limit = sqrt(R) + 1;
```

```

getSmallPrimes(limit);

// 用每个小素数分解区间内的数
for (int p : primes) {

    ll start = (L + p - 1) / p * p;
    for (ll j = start; j <= R; j += p) {
        int idx = j - L;
        ll exp = 0;

        while (val[idx] % p == 0) {
            val[idx] /= p;
            exp++;
        }

        if (exp > 0) {
            // 等比数列求和:  $1 + p + p^2 + \dots + p^{(exp \cdot K)}$ 
            //  $= (p^{(exp \cdot K + 1)} - 1) / (p - 1)$ 
            ll power = qpow(p, exp * K + 1, MOD);
            ll sum = (power - 1 + MOD) % MOD;
            ll inv = qpow(p - 1, MOD - 2, MOD); // 费马小定理求逆元
            sigma[idx] = sigma[idx] * sum % MOD * inv % MOD;
        }
    }
}

// 处理剩余的大质因子 (> sqrt(R))
for (int i = 0; i <= len; i++) {
    if (val[i] > 1) {
        // 单独一个质因子 p, 指数为 1
        ll power = qpow(val[i], K + 1, MOD);
        ll sum = (power - 1 + MOD) % MOD;
        ll inv = qpow(val[i] - 1, MOD - 2, MOD); // 费马小定理求逆元
        sigma[i] = sigma[i] * sum % MOD * inv % MOD;
    }
}

// 此时  $\sigma[i] = \sigma((L+i)^K) \bmod MOD$ 
}

```

代码解读:

1. **等比数列求和**：对于质因子 p 的指数 a ，在 n^K 中指数变为 $a \times K$ ，贡献为：

$$1 + p + p^2 + \cdots + p^{a \times K} = \frac{p^{a \times K + 1} - 1}{p - 1}$$

2. **逆元**：由于模数是质数（如 $10^9 + 7$ ），用费马小定理求 $p - 1$ 的逆元。

3. **最后处理**：若 $\text{val}[i] > 1$ ，它是一个大质数，指数为 1，贡献为 $\frac{p^{K+1}-1}{p-1}$ 。

注意事项

- **模数选择**：若模数为质数（如 $10^9 + 7$ ），可用费马小定理求逆元。若模数非质数，需用 exgcd 求逆元。
- **数值范围**： $p^{a \times K + 1}$ 可能极大，快速幂时需对 MOD 取模。
- **$p - 1$ 为 0 的情况**： $p = 1$ 不存在，无需担心。

习题

题号	名称
Luogu P1835	素数密度
HDU 6069	Counting Divisors
UVA 294	Divisors
EOlymp 1302	Almost Prime Numbers

模块七：综合

一、经典题讲练

题1：UVA 10375 Choose and Divide

题意：输入 p, q, r, s ($\leq 10^4$)，计算 $\frac{C(p,q)}{C(r,s)}$ ，结果 $< 10^8$ ，保留5位小数。

知识点：唯一分解定理

分析思路：

1. 组合数公式： $C(n, m) = \frac{n!}{m!(n-m)!}$

2. 原式 = $\frac{p!}{q!(p-q)!} \times \frac{s!(r-s)!}{r!}$

3. 用数组 $\text{exp}[i]$ 记录第 i 个素数的指数，乘的时候 +1，除的时候 -1

4. 最后 $\text{ans} = \prod p_i^{\text{exp}[i]}$

代码框架：

```
void addFact(int n, int sign) { // sign=1 乘, sign=-1 除
    for (int i = 1; i <= n; i++) {
        int x = i;
        for (int j = 0; j < cnt && primes[j] <= x; j++) {
            while (x % primes[j] == 0) {
                exp[j] += sign;
                x /= primes[j];
            }
        }
    }
}
// 调用: addFact(p,1); addFact(q,-1); addFact(p-q,-1);
//       addFact(r,-1); addFact(s,1); addFact(r-s,1);
```

题2: UVA 10791 最小公倍数的最小和

题意：输入 n ($1 \leq n < 2^{31}$)，求至少两个正整数，使它们的最小公倍数为 n ，且和最小，输出最小和。

知识点：唯一分解定理

分析思路：

1. 对 n 质因数分解： $n = p_1^{a_1} \times p_2^{a_2} \times \cdots \times p_k^{a_k}$
2. **关键结论**：最优方案是把每个 $p_i^{a_i}$ 单独作为一个数
3. 因为若把两个质因子的幂合并，会增大最小公倍数或者增大和
4. 特殊处理：若 $n = 1$ ，答案为 2（取 1 和 1）；若 n 为质数或质数幂，需要补一个 1

代码：

```
long long solve(int n) {
    if (n == 1) return 2;
    long long ans = 0;
    int cnt = 0;
    for (int i = 2; 1LL * i * i <= n; i++) {
        if (n % i == 0) {
            int pk = 1;
            while (n % i == 0) { pk *= i; n /= i; }
            ans += pk;
            cnt++;
        }
    }
    if (n > 1) ans += n;
    return ans;
}
```

```

    }
}
if (n > 1) { ans += n; cnt++; }
if (cnt == 1) ans++; // n 是质数或质数幂, 补一个 1
return ans;
}

```

题3: Luogu P4549 【模板】裴蜀定理

题意: 给定 n 个数 $A_1, \dots, A_n$, 求整数序列 X_i 使 $S = \sum A_i X_i > 0$ 且 S 最小。

知识点: 裴蜀定理

分析思路:

1. 裴蜀定理: $\sum A_i X_i$ 能取到的所有值构成集合 $\{k \times \gcd(A_1, \dots, A_n) \mid k \in \mathbb{Z}\}$
2. 最小的正数就是 $\gcd$ 的绝对值
3. 答案 = $\gcd(|A_1|, |A_2|, \dots, |A_n|)$

易错点: A_i 可能为 0, 需跳过; 序列不全为 0。

题4: Luogu P2155 [SDOI2008] 沙拉公主的困惑

题意: T 组数据, 给定 N, M 和质数 R , 求 1 到 $N!$ 中与 $M!$ 互质的数的个数对 R 取模。

知识点: 欧拉函数、线性筛、逆元

分析思路:

1. 答案 = $\frac{N!}{M!} \times \varphi(M!)$
2. $\varphi(M!) = M! \times \prod_{p \leq M} (1 - \frac{1}{p})$
3. 预处理:
 - $\text{fact}[i] = i! \bmod R$
 - $\text{inv}[i] = i$ 的逆元
 - $\text{ans}[i] = \text{ans}[i-1]$, 若 i 是质数则乘以 $(i-1) \times \text{inv}[i]$
4. 最终答案 = $\text{fact}[N] * \text{ans}[M] \% R$

数据范围: $N, M \leq 10^7$, 需用线性筛预处理。

思维拓展题

题号	名称	知识点	难度
Luogu P3938	Fibonacci	矩阵快速幂 + 斐波那契性质	☆☆